

Stacks

Notes on the ADT, both implementations, and its classic applications, with Java and C/C++ examples

A note before you begin. A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. While C/C++ was my first language, I am a bit rusty because I have taught Java for years now. For these notes, I have been using AI to help type these up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are not meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing students and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. We are a two-language course, thus I will do my best to show everything in Java (CIS) and C/C++ (CS).

1. The stack ADT

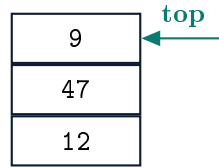
A stack is an abstract data type, and the wall between what it offers and how it is built behind the scenes works the same way it did for the linked list, covered in that unit's notes. We will not repeat that discussion here, only apply it. A stack ADT specifies a small set of operations, push, pop, peek (or top), isEmpty, and size, without saying anything about how those operations are carried out underneath. This unit covers two ways to carry them out: an array-based version and a linked version.

2. Terminology

- **LIFO.** Last in, first out. The most recently added item is the first one removed.
- **Top.** The end of the stack where every push and pop happens. The other end of the stack, sometimes called the bottom, is never touched directly.
- **Push.** Adds a new item at the top.
- **Pop.** Removes and returns the item at the top.
- **Peek (or top).** Returns the item at the top without removing it.
- **Underflow.** An attempt to pop or peek an empty stack. Every implementation needs to guard against this.
- **Overflow.** An attempt to push onto a stack that has reached a fixed capacity. This only applies to an array-based stack that does not resize itself.

3. Array-based versus linked implementation

An array-based stack keeps its items in a contiguous block and tracks the top with an index. A linked stack keeps its items in nodes, the same node and link idea from the linked list unit, and



bottom of stack

Figure 1: A stack holding three items. Every push and pop happens at the top; the bottom of the stack is never touched directly.

tracks the top with a reference to the first node in the chain.

Table 1: Array-based versus linked implementations of a stack.

Aspect	Array-based	Linked
Memory layout	Contiguous block, plus a top index	Scattered nodes, top is a reference to the first node
Capacity	Fixed, or resized (typically doubled) once full	Grows and shrinks one node at a time, no fixed limit
Push cost	$O(1)$, occasionally $O(n)$ when a resize is triggered	$O(1)$
Pop cost	$O(1)$	$O(1)$
Overflow	Possible if fixed-size and full	Not applicable, bounded only by available memory

A note on the code. The Java and C/C++ snippets below are not the exact code in your textbook, and I expect the two to look different in places. I simplify and reorganize for teaching purposes, and the book may name methods differently, split the class up differently, or handle a case, such as resizing or an empty stack, in a different order. These are snippets, not full listings, meant to show the key idea in each operation, not a complete class you can compile as-is. When the versions genuinely disagree, the textbook and the compiler settle it, not these notes.

4. Code snippets: array-based stack

The array-based stack keeps an array of items and an integer, often called top, that marks the index one past the last used slot (equivalently, the current size of the stack).

Java

```
private Object[] items;
private int top;    // number of items currently on the stack

public void push(T value) {
    if (top == items.length) {
        items = Arrays.copyOf(items, items.length * 2);    // resize, doubling capacity
    }
    items[top] = value;
    top++;
}
```

```

}

@SuppressWarnings("unchecked")
public T pop() {
    if (isEmpty()) {
        throw new NoSuchElementException("stack is empty");
    }
    top--;
    T value = (T) items[top];
    items[top] = null;    // avoid holding a reference the stack no longer owns
    return value;
}

```

C/C++

```

T* items;
int capacity;
int top;    // number of items currently on the stack

void push(const T& value) {
    if (top == capacity) {
        resize(capacity * 2);    // allocate a bigger array and copy items over
    }
    items[top] = value;
    top++;
}

T pop() {
    if (isEmpty()) {
        throw std::underflow_error("stack is empty");
    }
    top--;
    return items[top];
}

```

5. Code snippets: linked stack

The linked stack keeps a single reference, `top`, pointing at the first node in the chain. Pushing and popping both happen at that one reference, so neither operation ever has to walk the list.

Java

```

private Node<T> top;

public void push(T value) {
    Node<T> newNode = new Node<>(value);
    newNode.setNext(top);    // new node points at the old top
    top = newNode;           // top now points at the new node
}

```

```

public T pop() {
    if (isEmpty()) {
        throw new NoSuchElementException("stack is empty");
    }
    T value = top.getData();
    top = top.getNext();
    return value;
}

```

C/C++

```

Node<T>* top;

void push(const T& value) {
    Node<T>* newNode = new Node<T>(value);
    newNode->next = top;    // new node points at the old top
    top = newNode;        // top now points at the new node
}

T pop() {
    if (isEmpty()) {
        throw std::underflow_error("stack is empty");
    }
    Node<T>* oldTop = top;
    T value = oldTop->data;
    top = top->next;
    delete oldTop;        // free the node we just unlinked
    return value;
}

```

6. Applications, in pseudocode

Both classic applications below only need push, pop, peek, and isEmpty, which is part of why the stack ADT shows up so often once you know to look for it.

Checking for balanced parentheses

```

create an empty stack
for each character c in the input, left to right:
    if c is an opening symbol ( ( [ { ):
        push c
    else if c is a closing symbol ( ) ] } ):
        if the stack is empty:
            return "not balanced"
        pop the top symbol and check that it matches c
        if it does not match:
            return "not balanced"
after the loop, if the stack is empty:
    return "balanced"
else:

```

```
return "not balanced"    // some opening symbol was never closed
```

Evaluating a postfix expression

Postfix notation (also called reverse Polish notation) writes an operator after its two operands, for example `3 4 +` instead of `3 + 4`, which avoids any need for parentheses or operator precedence rules.

```
create an empty stack
for each token t in the postfix expression, left to right:
    if t is a number:
        push t
    else if t is an operator (+, -, *, /):
        pop the top value into b
        pop the top value into a
        compute result = a t b    // for example, a + b if t is "+"
        push result
after the loop, pop and return the single remaining value on the stack
```

7. Stacks and recursion

The call stack that a program uses to keep track of recursive calls, from the first unit of this course, is a stack in exactly this sense: each call pushes a new frame holding that call's local variables and its return address, and returning from a call pops the corresponding frame back off. Every recursive algorithm can be rewritten as a loop that manages its own explicit stack instead of relying on the language's call stack, which is one reason the stack ADT and recursion keep showing up together.

8. Traps people fall into

- **Popping or peeking an empty stack.** Always check `isEmpty` first, or catch the exception the operation throws. This is the single most common stack bug.
- **Off-by-one on the top index.** Decide once whether an array-based stack's top index points at the last used slot or at the next free slot, and be consistent. Mixing the two conventions across push and pop is a frequent source of bugs.
- **Forgetting to resize (or checking too late).** An array-based stack that does not check capacity before writing will overflow the array's bounds instead of reporting overflow cleanly.
- **Leaving a stale reference in Java.** After popping from an array-based stack, the old slot still refers to the popped object unless you explicitly clear it, which can keep that object alive longer than intended.
- **Forgetting to delete in C/C++.** Popping a node from a linked stack does not free its memory by itself. If you are not using smart pointers, you still need to call `delete` on the node you just unlinked, or the memory leaks.

9. Where this lives in the textbooks

Both books use the wall to name the ADT chapter, so it is easy to find.

- **Java, 3rd edition (Prichard and Carrano).** Chapter 7, Stacks, covers the ADT itself (7.1), simple applications such as checking for balanced braces (7.2), three implementations, array-based, reference-based, and one built from the ADT list, plus a comparison and the Java Collections Framework's own Stack class (7.3), postfix evaluation and infix-to-postfix conversion (7.4), a search-problem application (7.5), and the relationship between stacks and recursion (7.6).
- **C/C++, 8th edition (Carrano and Henry).** Chapter 8, Stacks, covers the ADT, simple uses of a stack, using a stack to evaluate postfix expressions, and the relationship between stacks and recursion. Chapter 9, Stack Implementation, covers an array-based implementation, a link-based implementation, and a version built with exceptions.